Khalani: A Next-Generation Blockchain with On-Chain Proofs

The Khalani Labs Team

31 October 2025

1 The main part of the litepaper comes here

Appendices

A Overview of Axi

Axi (pronounced "Axee") is the smart contract language of Khalani. It is a purely functional programming language with a strong, static, linear type system and support for formal reasoning about programs in classical higher-order logic. Among mainstream programming languages, it is most similar to Haskell (in spirit) and OCaml (syntactically). It was also heavily inspired by proof assistants based on type theory, like Coq, Lean and Agda.

In this appendix we will have a brief, not too technical look at the philosophy and language features of Axi, together with a few simple code examples.

A.1 Philosophy

Axi's architecture is unique in that the language is split into two strictly separate layers: the programming layer and the logic layer.

The Programming Layer of Axi is a purely functional language based on System F with quantitative typing (linear, affine, relevant and unrestricted types) and algebraic data types. It aims to make programming pleasant and comprehending smart contract code easy, while providing essential guarantees to all on-chain code:

- **Type safety**. Well-typed programs can't go wrong, i.e. they can't crash (unless they run out of gas), can't perform nonsensical operations like adding a number and a string, and can't result in any other runtime type error.
- **Memory safety**. Well-typed programs can't result in memory errors, like writing to a random memory location, nor can they have memory leaks.

- **Resource safety**. Well-typed programs can't access resources they don't have permission to, can't delete tokens that are not supposed to be deletable and can't create money out of thin air.

The Logic Layer of Axi is a language for reasoning about Axi programs in classical higher-order logic. It allows expressing arbitrarily complex mathematical properties, thus giving programmers the ability to precisely specify the business logic and behaviour of their programs. These specifications can then be either tested with random inputs before deployment, checked for concrete inputs at runtime, or proved for all possible inputs using logic. The last of these, proving, provides guarantees much more powerful than those of the type system alone, while being as credible and trustworthy as the type system. The use of the logic layer is entirely optional, i.e. programmers are never forced to engage with any part of it, although competition will encourage programmers to do so in the hopes of outcompeting other programmers' smart contracts by providing stronger guarantees.

Strict separation. In Axi, logic can reason about programs, but programs cannot manipulate logical entities, like theorems or proofs, at runtime. This one-way interaction keeps the language relatively simple, while still making it possible to express almost all functional programming idioms and all possible mathematical properties and their proofs.

Guarantees about guarantees. We not only want Axi to provide strong guarantees about its programs, but we also want to provide strong guarantees about Axi itself. We plan to use the Coq proof assistant to formally verify significant portions of Axi's metatheory, including the aforementioned properties of type and resource safety, as well as the consistency of Axi's logic. We are also considering formal verification of the reference implementation of Axi. The goal of these efforts is to ensure that the guarantees that Axi provides are trustworthy and will stay so forever, as long as the consensus protocol is followed.

A.2 Language features

In this section we will take a closer look at Axi's language features.

A.2.1 The Programming Layer

Axi is, at its core, a purely functional programming language. It thus supports all the usual features of such languages:

- **Purity amd immutability**. In Axi, all values are pure and immutable – they cannot be null, cannot throw exceptions and once created, they cannot be changed. This facilitates reasoning about and comprehending code, thus rendering Axi smart contracts transparent and auditable.

- **First-class, higher-order and anonymous functions**. Functions are first class citizens. They can be passed as arguments to other functions, returned from other functions as results and created on the fly. Axi encourages a programming style in which data is processed by pipelines built from basic building blocks like mapping, reducing, filtering and so on.

- **Algebraic data types and exhaustive pattern-matching**. Instead of clunky boolean flags and cascades of ifs, Axi makes it easy to model

the problem domain with disjoint unions and define functions on them by pattern matching and explicitly handling all possible cases.

- **Strong static type system**. Axi's type system makes it easy to work with high-level abstractions, like merkle trees or state machines, instead of low-level primitives, like integers or byte arrays. It is lightweight, not placing too many demands on the programmer, but still catches the most egregious errors – it can easily protect programmers from confusing with each other such objects as an integer, a string, an address, a transaction hash, a token amount and a cryptographic secret.

- **Polymorphism and traits**. Axi's type system features polymorphism (also known as "generics") as well as traits inspired by Haskell's type classes (but named after Rust's traits). These features make it easy to implement data structures and functions that work for all types that support a certain behaviour, for example a hash map that works for all types of ERC-20-compliant tokens bridged from an external blockchain.

Quantitative typing. The standard features of purely functional programming is not everything that Axi has to offer. The killer feature, as far as the programming layer is concerned, are quantitative types, which allow the type system to enforce correct resource usage discipline. This aspect of Axi's type system was inspired by linear logic and its cousin, quantitative type theory, and is an improvement upon an analogous feature of the type system of Move, the programming language of the Sui and Aptos blockchains.

In an Axi program, every variable is assigned its *quantity*, which can be either: zero, one, at most one, at least one, unrestricted. The variable's quantity determines how many times it can be used in the program, with too many or too few uses resulting in a type error. For example, the typical variable holding an NFT will have a quantity of one, forcing the program to do something with the NFT before it terminates: maybe send the NFT to the transaction's originator, maybe trade it for something else and handle that something else properly according to its quantity, or maybe send it to somebody as a gift. What the program cannot do in such a situation is forget the NFT and do nothing with it, effectively making it "lost" in the guts of the chain. It also cannot copy the NFT and have two instead of just the original one.

To make the programmers' lives easier, every type is also assigned its "default" quantity. Every variable will be assigned the default quantity of its type unless the programmer decides otherwise. An important piece of terminology: a type whose default quantity is unrestricted is called a *data type*, whereas a type whose default quantity is either one, at least one or at most one, is called a *resource type*. There are no types whose default quantity is zero. Elements of data types are called data, whereas elements of resource types are called resources.

Traits. A trait is a way of representing an interface satisfied by a type. The definition of a trait consists of a bunch of declarations of constants and functions. The programmer can make a type an *instance* of a trait by implementing the trait's constants and functions for this type.

A.2.2 The Logic Layer

Axi's logic layer consists of typed classical higher-order logic with equality whose domains of discourse are the types from the programming layer. More precisely, this means the logic has:

- **Propositional connectives and constants**. Standard propositional connectives ($\implies$, $\wedge$, $\vee$, $\iff$, $\sim$) and constants ($\top$, $\bot$) can be used to build up complex propositions from simpler ones.
- **Quantification over terms and types**. Axi supports universal and existential quantification. We can quantify over terms of a given type ($\forall x : A, \dots$) as well as over types themselves ($\forall A : \texttt{Type}, \dots$), no matter whether we're dealing with data, resources or programs.
- **Higher-order quantification**. We can also quantify over logical entities like propositions ($\forall P : \texttt{Prop}, \dots$), predicates ($\forall P : A \to \texttt{Prop}, \dots$), relations ($\forall R : A \to A \to \texttt{Prop}, \dots$), logical connectives ($\forall C : \texttt{Prop} \to \texttt{Prop} \to \texttt{Prop}, \dots$) or even quantifiers themselves ($\forall Q : (A \to \texttt{Prop}) \to \texttt{Prop}, \dots$). This ability of logic to talk not only about the underlying programming language, but also about itself, gives it great expressive power and allows it to make use of all of mathematics when reasoning about the behaviour of programs.
- **Equality**. The built-in equality relation ($=$) makes it possible to precisely reason about programs' input-output behaviour.

Proving is carried out in a proof language based on the Curry-Howard correspondence: proofs of implications and universal quantifiers are like functions, proofs of existential quantifiers and conjunctions are like pairs, proofs of disjunctions are like disjoint union constructors, etc. Negation and biconditional are handled as in intuitionistic logic - instead of being primitives, negation ($\sim P$) is a shorthand for an implication into falsehood ($P \implies \bot$), whereas biconditional ($P \iff Q$) is a shorthand for a conjunction of two implications ($(P \implies Q) \wedge (Q \implies P)$). The principle of *proof by contradiction*, which allows to prove $((\sim P \implies \bot) \implies P)$ for any proposition P, makes the logic classical and thus intimately familiar to all mathematicians.

Equality is handled as in Martin-Löf Type Theory – proved by reflexivity and eliminated with the so-called J rule, which is a principle that allows to substitute equals for equals in the remaining part of the proof. Because proofs are hard to write using these primitives, Axi provides a convenient feature called *chaining*, which allows reasoning with long chains of equations by writing down and justifying each step separately, similarly to how equality is usually handled with pen and paper. Besides equality, this feature also supports implications, biconditionals and other transitive relations.

One novel feature that distinguishes our proof language is that the proofs can be presented in two styles: either in a *proof term style*, similar to how proofs look in Agda and Idris (as well as Coq's and Lean's manually written proof terms), or in a *tactic style*, similar to how proofs usually look in Coq or Lean. The difference between Axi and languages that also support both styles, like Coq and Lean, is that in these languages each style is its own separate proof language, whereas in Axi there is a single proof language which, thanks to a clever parser,

can be formatted either like terms or like tactics. This feature will make it easier to accomodate computer scientists and mathematicians familiar with either of the aforementioned languages.

Concepts – traits in logic. Last but not least, the logic can not only talk and reason about the traits we met in the programming layer – it also introduces the richer and more powerful construct called *concepts*. Concepts extend traits with specifications, i.e. properties expected to be true of the trait's constants and functions. When implementing a concept, the programmer must also provide proofs of the specifications. Thus concepts let programmers precisely define what is expected of trait implementations.

B Code examples

In this section we will see some code examples that demonstrate how Axi works in practice. Note that Axi is not implemented yet, so that the code examples are only an approximation of what we intend to achieve, instead of being a demonstration of an existing language.

Example 1. *An example program – list functions*

```
1  data List (A : Type) : Type where
2    nil
3    cons (h : A) (t : List A)
4
5  app @(A : Type) (l1 l2 : List A) : List A =
6    match l1 with
7    | nil      => l2
8    | cons h t => cons h (app t l2)
9
10 map @A @B (f : A -> B) : List A -> List B
11 | nil      => nil
12 | cons h t => cons (f h) (map f t)
```

The above example demonstrates the basics of Axi programming. In lines 1-3 we define an algebraic data type called `List`. The type is parameterized by a type `A` and has two constructors: `nil` is the empty list and `cons h t` is the list with head `h` and tail `t`. Both annotations `: Type` are necessary, because we need to indicate that both `A` and `List A` are unrestricted types, as opposed to linear, affine or relevant (we will see these in later examples).

In lines 5-8 we define a function called `app` which concatenates two lists. It takes three arguments: a type `A` and two lists `l1` and `l2` of type `List A`, and returns a result of type `List A`. The type argument means that the function is polymorphic. It is also marked with the symbol `@`, which means it's an *implicit* argument, i.e. when calling the function it can be omitted and will get automatically inferred. The body of the function is simple: we pattern match on the argument `l1`. If it's `nil`, we return `l2`. If it's of the form `cons h t`, the result's head is `h` and its tail is computed by the recursive call `app t l2`.

In lines 10-12 we define another function, called `map`, which applies a function `f` to every element of a list. This time, however, we used an abbreviated syntax. The two type arguments `@A` and `@B` are marked as implicit, but we don't write `: Type`, which gets inferred automatically. The argument `f` and its type are written out in full. The last argument, of type `List A`, and the result type `List`

B, appear after the last `:` as the type `List A -> List B`. Because of this, it is understood that in the next, pattern matching on the input of type `List A` follows.

Example 2. *Quantitative typing*

```
// ExampleCoin : Type1 is a resource type that we'll define later.

coinForgery (x : ExampleCoin) : ExampleCoin = coinMerge x x
//           ^                                            ^ ^
//           |                                            | |
//           \-------------------------------------\      | |
//                                                 |      | |
// Type error: duplicate use of the resource 'x' -/      | |
// of type 'ExampleCoin : Type1'                          | |
// First use ------------------------------------------/ |
// Second use -----------------------------------------------/
// Hint: remove the duplicate use
// or declare the type to be in 'Type+' or 'Type' to allow it

coinBlackhole (x : ExampleCoin) : Unit = ()
//             ^                         ^
//             |                         |
//             \--------------------\    |
//                                  |    |
// Type error: unused resource 'x' -/    |
// of type 'ExampleCoin : Type1'         |
// Hint: use the resource in body -------/
// or declare the type to be in 'Type?' or 'Type' to allow it
```

The above example demonstrates the kind of resource discipline enforced by the type system. We will define the type `ExampleCoin` and the function `coinMerge` in the next example. For now it only matters that `ExampleCoin` is a resource type, i.e. `ExampleCoin : Type1`.

First we try to define a function called `coinForgery`. It takes an argument of type `ExampleCoin` called `x` and returns an `ExampleCoin` by calling the function `coinMerge` with boths its arguments being `x`. Now, `ExampleCoin` is a linear type, i.e. by default variables of this type must be used exactly once. Because `x` is of type `ExampleCoin` and nothing indicates otherwise, this means that the function `coinForgery` must use `x` exactly once. However, it uses it twice when calling `coinMerge`, which is a type error.

This error is caught by the type checker, which complains that `x` was used too many times. The error message also hints at some possible solutions: use `x` less (the only solution for ordinary programmers) or redefine the type `ExampleCoin` to allow duplicate uses (this solution is only available to the type's implementer). The fact that the type system was able to catch this error is crucial, because if the function `coinForgery` passed the type checker, it would allow programmers to create unbounded amounts of `ExampleCoin`, causing hyperinflation, which most likely was not the intention of this type's creator.

The second function, called `coinBlackhole`, takes an argument `x` of type `ExampleCoin` and returns a value of type `Unit`, which is a type with only one element `()`. The `Unit` type is returned to indicate that the function's result "doesn't matter". Because the type of `x` is linear, `coinBlackhole` must use `x` exactly once. However, it doesn't use it at all, which is a type error.

This error is caught by the type checker, which complains that `x` was not used. Similarly to the previous error message, two solutions are offered: use `x`

or make the type `ExampleCoin` affine so that it allows zero or one usages. The fact that this error was caught by the type system is very important, because `ExampleCoin`'s creator did not intend for values of this type to be deletable.

Example 3. *Coin trait*

```
1  trait Coin (A : Type1) where
2    coinAmount : A -> Nat * A
3    coinZero   : A
4    coinMerge  : A -> A -> A
5    coinSplit  : Nat -> A -> Option A * A
6
7  record ExampleCoin : Type1 where
8    amount : Nat
9
10 instance Coin ExampleCoin where
11   coinZero : ExampleCoin =
12     record where amount = 0
13
14   coinMerge (c1 c2 : ExampleCoin) : ExampleCoin =
15     record where amount = c1.amount + c2.amount
16
17   coinAmount (c : ExampleCoin) : Nat * ExampleCoin =
18     let n : Nat = c.amount in
19       (n, record where amount = n)
20
21   coinSplit (askedAmount : Nat) (c : ExampleCoin) :
22     Option ExampleCoin * ExampleCoin =
23       let n : Nat = c.amount in
24         if askedAmount <= n
25         then
26           ( some (record where amount = askedAmount),
27           , record where amount = n - askedAmount)
28           )
29         else
30           (none, record where amount = n)
```

The above example shows the definition of the `Coin` trait, which could be one way of capturing the commonalities of user-defined coins. Then we implement the type type `ExampleCoin` from the previous example and demonstrate how to make it an instance of the `Coin` trait.

In lines 1-5 we define the trait `Coin`. It takes a parameter `A : Type1`, i.e. this trait can only be implemented by resource types. The trait has one constant and three functions. `coinZero` represents a coin with denomination zero. `coinMerge` merges two coins. `coinSplit` splits one coin into two according to the amount given in its first argument, with the possibility of failure (if the amount asked is too big) indicated by `Option` in the return type. `coinAmount` is perhaps the most mysterious – it returns the denomination of the coin, but also a new coin. This is a pattern that we will see many times to come. If `coinAmount` returned just the amount, that would destroy the coin, because it must be used exactly once (because its type is linear), and checking the amount counts as a single use! Therefore, `coinAmount` must not only return the amount, but also a "new" coin, which we expect to be the same as the "old" coin (but this is not guaranteed; later we will see how to guarantee this fact using logic).

In lines 7-8 we implement the type `ExampleCoin`. It is a record type with a single field `amount` which is a natural number (i.e. a non-negative number

which can be arbitrarily large). `ExampleCoin` is a resource type, as marked with the annotation `: Type1`.

In lines 10-27 we implement the trait `Coin` for the type `ExampleCoin`. Note that we don't need to implement the trait's constants and functions in the same order they appear in the trait definition. In lines 11-12 we implement `coinZero` – it's a record (of type `ExampleCoin`) whose `amount` field is `0`.

In lines 14-15 we implement `coinMerge`. It takes two arguments of type `ExampleCoin` and returns an `ExampleCoin` which is a record whose `amount` field is the sum of the `amount`s of its arguments.

In lines 17-19 we implement `coinAmount`. It takes an argument `c` of type `ExampleCoin` and returns a tuple whose first component is a natural number and whose secnd component is an `ExampleCoin`. First, in line 18, we consume the coin `c` by peeking at its `amount` field, which then gets named `n`. In line 19, we build the return tuple – the first component is just `n`, whereas the second component is a record whose `amount` field is `n`. Note that the latter is exactly the same thing as the initial coin `c`, but we cannot directly return `c` as the secnd component, because it must be used exactly once and we have already used it when checking its `amount`.

Last but not least, in lines 21-30 we implement `coinSplit`. It takes two argument, `askedAmount` and `c` of type `ExampleCoin`. First, in line 23, we consume `c` to check its amount and store it in `n`. Then on line 24 we check if `n` is big enough for the split to succeed. If it does, on lines 26-28 we return a tuple whose first component is a coin whose amount is the `askedAmount` (wrapped in `some` to indicate success) and whose second component is a coin whose amount is `n` minus the `askedAmount` that went to the other coin. If `n` is not big enough, we return a tuple whose first component is `none` (to indicate failure) and whose second component is a coin whose amount is `n`, i.e. a coin that is the same as the original coin `c`.

Example 4. *A simple proof*

```
1  theorem app-assoc :
2    forall (A : Type) (l1 l2 l3 : List A),
3      app (app l1 l2) l3 === app l1 (app l2 l3)
4  proof
5    pick-any A l1
6    induction l1 with
7    | nil =>
8      pick-any l2 l3
9      refl
10   | cons h (t & ind IH) =>
11     pick-any l2 l3
12     chaining
13       === app (app (cons h t) l2) l3
14       === cons h (app (app t l2) l3)
15       === cons h (app t (app l2 l3))  by rewrite IH
16       === app (cons h t) (app l2 l3)
17 qed
```

The above example shows a simple theorem and its proof. The theorem's name, `app-assoc` (which is a kind of mnemonic for "(list) append is associative") is in line 1. Its statement, in lines 2-3, reads as follows: for all types `A` and for all `l1`, `l2`, `l3` which are of type `List A`, the list `app (app l1 l2) l3` is equal to the list `app l1 (app l2 l3)`.

The proof is on lines 4-17. At any point in the proof, Axi keeps track of all the *goals* that need to be solved to finish the current proof, and also of the *context* of these goals, which is the list of all local objects and assumptions we can use in the proof. When we start the proof with the keyword `proof`, we start with a single goal that is the same as the theorem statement. The code on line 5 could be read "let `A` be any type and let `l1` be any list of `A`s". At this point the goal becomes `forall (l2 l3 : List A), app (app l1 l2) l3 === app l1 (app l2 l3)`. On line 6 we indicate that the rest of the proof proceeds by induction on the list `l1`. The induction has two cases, `nil` and `cons`, corresponding to the two list constructors. This means that we now have two (sub)goals, one for each case.

On line 7 we indicate the start of the (sub)proof for the first goal, i.e. the `nil` case. At this point our current goal is `forall (l2 l3 : List A), app (app nil l2) l3 === app nil (app l2 l3)`, because `l1` was replaced with `nil`, and after line 8 it is `app (app nil l2) l3 === app nil (app l2 l3)`. We finish the first subgoal on line 9. `refl`, which is short for "reflexivity", indicates that this equation holds trivially, just by computing the result on both sides.

On line 10 we indicate the start of the (sub)proof for the second goal, i.e. the `cons` case. On the same line we bind the arguments of `cons`, which we name `h` and `t`, as well as get hold of our induction hypothesis for the tail `t`, which we name `IH`. After line 11, the current goal is `app (app (cons h t) l2) l3 === app (cons h t) (app l2 l3)`, as `l1` was replaced with `cons h t`.

In lines 12-16, we use chaining to finish the goal off. It works like this: we start with the left hand side of the current goal (which must be an equation), which in our case is `app (app (cons h t) l2) l3` (line 13). Then in subsequent lines we write out what this expression is equal to (after the equality symbol `===`) and provide justifications for why this is the case (after the keyword `by`). In our case, on line 14 indicate that our goal's left-hand side is equal to `cons h (app (app t l2) l3)`, but we omit the justification for this. We could have written `by refl`, but this is the most trivial justification possible, so Axi guesses this is what we mean when we omit a justification. Then, on line 15, we indicate that it is equal to `cons h (app t (app l2 l3))`, justifying it with `by rewrite IH`, i.e. we are making use of our induction hypothesis. This justification is crucial and cannot be omitted, because Axi cannot infer it on its own. Finally, on line 16, we indicate that our expression is equal to the right-hand side of our goal, again omitting the justification, because it is trivial. All in all, the entire chaining proves that `app (app (cons h t) l2) l3` is equal to `app (cons h t) (app l2 l3)`, which is exactly what our goal was. Thus the proof is finished, which we indicate on line 17 with the keyword `qed`.

Example 5. *Lawful coin concept*

```
concept LawfulCoin (A : Type1) extends Coin A where
  coinAmount' (x : A) : Nat =
    let (n, _) = coinAmount c in n

  law coinAmount-spec :
    forall @(x : A),
      let (_, y) = coinAmount x in x === y

  law coinZero-spec :
    coinAmount' coinZero === 0

```

```
12  law coinMerge-spec :
13    forall @(x y : A),
14      coinAmount' (coinMerge x y)
15        ===
16      coinAmount' x + coinAmount' y
17
18  law coinSplit-none :
19    forall @(askedAmount : Nat) @(coin coin' : A),
20      coinSplit askedAmount coin === (none, coin')
21       <->
22      (askedAmount <= coinAmount' coin) === false
23        /\
24      coin === coin'
25
26  law coinSplit-some :
27    forall @(askedAmount : Nat) @(coin exact change : A),
28      coinSplit askedAmount coin === (some exact, change)
29       <->
30      (askedAmount <= coinAmount' coin) === true
31        /\
32      coinAmount' exact === askedAmount
33        /\
34      coinAmount' change === coinAmount' coin - askedAmount
```

The above example shows a concept called `LawfulCoin`, which extends the trait `Coin` that we met previously. Note this distinction: the trait and its implementation are a part of the programming layer, whereas the concept and its implementation are part of the logic layer.

In lines 2-3, we define a function `coinAmount'` that is part of the `LawfulCoin` concept. This is a feature we have not mentioned before: both traits and concepts, besides declarations, can also contain (auxiliary) definitions. In our case, we need the function `coinAmount'` to avoid using the much more cumbersome `coinAmount` when stating the properties that lawful coins should satisfy. `coinAmount'` takes an argument of type `A` (of which we think as a coin, because it has the `Coin` trait) and returns only the first output of `coinAmount`, while ignoring its second output.

It might seem that `coinAmount'` ignoring the coin returned by `coinAmount` is a type error (zero uses instead of exactly one), but function definitions that belong to the logic layer play by different rules from ordinary functions. Most importantly, they don't need to adhere to the resource usage discipline. In other words, auxiliary functions defined solely to be used to state theorems can freely delete or copy resources, even it were a type error for an ordinary function to do so. This is not a problem from the point of view of ordinary programs – these "logic-only programs" cannot be run (they can only be reasoned about when proving theorems), so they can never do anything bad to on-chain assets.

Next, in lines 5-34, comes the main part of the concept's definition. In lines 5-7 we declare a law claled `coinAmount-spec` which specifies the behaviour of `coinAmount` – the second component of its output must be equal to its input. This is the guarantee missing from the `Coin` trait that we were expecting to hold. Every implementation of `LawfulCoin` will have to provide a proof that turns this law into a theorem.

Having given a specification of `coinAmount`, all the remaining laws will use `coinAmount'` only. In lines 9-10 we declare the law `coinZero-spec` which says that the amount of `coinZero` is zero. In lines 12-16, the law `coinMerge-spec`

says that the amount in a merged coin is equal to the sum of amounts of the individual coins.

In lines 18-24 the law `coinSplit-none` says that `coinSplit` returns `(none, coin')` if and only if the asked amount was not smaller than `coin`'s amount and the coins are equal. However, this only specifies one of the two cases that may happen when calling `coinSplit`, so we need another law. In lines 26-34 the law `coinSplit-some` says that `coinSplit` returns `(some exact, change)` if and only if the asked amount was smaller than `coin`'s amount, the amount of `exact` is equal to `askedAmount` and the amount of `change` is equal to `coin`'s amount minus the asked amount. Together, these two laws fully specify the working of `coinSplit`: if the coin's amount is enough, the coin gets split into a coin with the amount we asked and a coin which contains the "change". If we asked for too much, we just get the original coin back.

Example 6. *Implementation of `LawfulCoin` for `ExampleCoin`*

```
instance LawfulCoin ExampleCoin where
  theorem coinAmount-spec :
    forall @(x : ExampleCoin),
      let (_, y) = coinAmount x in x === y
  proof
    pick-any x
    refl
  qed

  theorem coinZero-spec :
    coinAmount coinZero === (0, coinZero)
  proof
    refl
  qed

  theorem coinMerge-spec :
    forall @(x y : ExampleCoin),
      coinAmount' (coinMerge x y)
        ===
      coinAmount' x + coinAmount' y
  proof
    pick-any x y
    refl
  qed

  // The proofs of coinSplit-none and coinSplit-some were elided.
```

The above example shows how to implement `LawfulCoin` for the type `ExampleCoin`. We don't need to reimplement `coinAmount'` or anything like that – this function is defined by the concept and we can use it freely. What we need to do is to replace every `law` from the concept's definition with a `theorem`. It so happens that we implemented `coinAmount`, `coinZero` and `coinMerge` correctly and also our implementations are so simple that the proofs go through just by computation (i.e. `refl`). The only theorems requiring a non-trivial amount of proving would be `coinSplit-none` and `coinSplit-some`, but we omit these proofs because they are more about natural numbers than the concept of `LawfulCoin`.

C Interacting with the ledger in Axi

In this appendix, we will see how Axi programs can interact with Khalani's ledger. First we will discuss a simplified ledger model, which we will use in the remaining code examples, and look at the primitive (buil-in) types and functions that represent it in Axi. Then we will give an overview of the big idea behind Axi's interaction with the ledger, which we might call the "resource lifecycle". Finally we will discuss a slightly longer example of an on-chain service, together with example transactions that set this service up and use it.

C.1 A simplified ledger model and Axi primitives

We will use the word *ledger* to refer to the state of the blockchain. Things on the ledger have their addresses. The three kinds of addresses are *accounts*, *package addresses* and *cell addresses* (or just addresses for short).

Accounts are addresses that act as the identities of external actors. They allow them to interact with the blockchain, similarly to EOAs (Externally Owned Accounts) on Ethereum.

All Axi code deployed onto the ledger is stored in a big codebase, divided into *packages*. Package addresses are means of locating these packages in this codebase. Packages are further subdivided into smaller units, called *modules*, which hold definitions of types, interfaces, constants and functions.

Data and resources on the ledger are stored in *cells*, also called *objects*. Each cell has an address and a type, and stores exactly one value of that type. Cells also have some metadata which won't concern us for now.

Objects are either *owned* or *shared*. Owned objects have exactly one owner who is an account. Shared objects do not have an owner. Shared objects are further subdivided into *mutable shared objects*, which contain resources and can be interacted with by anybody (any account or object) according to the interface set by the package defining the object's type, and *immutable shared objects*, which contain data but not resources and can be read by anybody, but not modified.

The state of the ledger is modified by transactions. Every transaction has a *transaction originator* and a *transaction body*, which is an Axi function that describes how to modify the ledger. Transactions also have some additional metadata which won't concern us here.

Example 7. *Axi primitives for interacting with the ledger*

```
primitive
  // provides access to ledger and transaction metadata
  Ledger : Type1

  // a unique address of a cell or an account
  Address : Type

  // an address that signed the currently executed transaction
  whoami : Ledger -> Address * Ledger

  // create a new cell owned by a specified address
  new : forall (A : Type1), Address -> Ledger -> Address * (A -> Ledger)

  // destroy an existing cell owned by transaction signer
  delete : forall @(A : Type1), Address -> Ledger -> Option A * Ledger
```

```
// reference to a shared cell
Ref : Type1 -> Type

// create a new mutable shared cell, accessible by everyone
share : forall @(A : Type1), Ledger -> Ref A * (A -> Ledger)

// load contents of a shared cell and then store a new value
update : forall (A : Type1), Ref A -> Ledger -> A * (A -> Ledger)

// upcast a reference to an address
addressof : forall (A : Type1), Ref A -> Address

// downcast an address to a reference
referenceof : forall @(A : Type1),
  Address -> Ledger -> Option (Ref A) * Ledger

// create a new immutable shared cell, accessible by everyone
freeze : forall (A : Type), A -> Ledger -> Address * Ledger

// read contents of an immutable cell
peek : forall @(A : Type), Address -> Ledger -> Option A * Ledger
```

The above code listing shows the most important primitives (built-ins) available in Axi. `Address` is the data type of addresses, including accounts. `Ledger` is an abstract type that represents (the current state of) the ledger. The body of a transaction is a function of type `Ledger -> Ledger`.

Every function that interacts with the ledger takes (the current version of) the ledger as input and also returns (the new version of) the ledger as output. In general, the type of such a function is `A -> Ledger -> B * Ledger`, i.e. it takes a value of some type `A` and the ledger, and returns a value of some type `B` and the (updated) ledger.

The transaction body will, in general, be composed of many such functions. It will get the initial input of type `Ledger` from the transaction context and then pass it to the first of these functions, which will in turn pass it to the second one, and so on. Because `Ledger` is a resource type, the type system guarantees that values of this type cannot be copied or deleted. This means that the ledger is acted upon and then passed further and further until the transaction body returns the final modified version of it.

An example function that needs the ledger is `whoami`, which consumes the old ledger and returns the account address that is the originator of the current transaction, together with a new ledger.

`new` is a function that takes an account address and a value of some type, together with the ledger, creates a new cell on the ledger that contains this value and belongs to that account, and returns the address of the created cell together with the new ledger.

`delete` is a function that takes a cell address, together with the ledger, and returns an optional value of some type `A`, together with the new ledger. If the transaction originator is not the owner of the cell, or the value stored in the cell is not of type `A`, then the operation fails and the result is `none`. Otherwise the operation succeeds, the cell is destroyed and the result is `some a`, where `a` is the value that was stored in this cell.

`share` is a function that takes a value of some type `A`, together with the ledger, creates a new mutable shared cell that contains this value, and returns

the reference to the created cell together wtih the new ledger.

`freeze` is a function that takes a value (which must be data and not a resource), together with the ledger, creates a new immutable shared cell that contains this value, and returns the address of the created cell together with the new ledger.

`peek` is a function that takes an address, together with the ledger, reads the contants of an immutable shared cell under this address, and returns an optional value of some type `A` together with the new ledger. If the address does not point to an immutable shared cell, or the value stored in that cell is not of type `A`, then the operation fails and the result is `none`. Otherwise the operation succeeds and the result is `some a`, where `a` is the data stored in the cell.

`update` is a function that takes a reference to a shared cell, together with the ledger, and returns a value of some type `A` together with a callback that when called with the new value of the cell, returns the new ledger. Note that this callback has a resource type `A -> Ledger`, so the type system makes sure that the callback is eventually called once.

C.2 The resource lifecycle

A new resource type is created by a transaction that deploys a package of Axi code that contains the definition of this resource type. The only exception is Gas, the native coin used to pay transaction fees, which is built-in, so it already exists at the origin block. Because packages live on-chain forever and cannot be deleted, a resource type, once it comes to life, will also exist forever.

The life cycle of a particular resource, i.e. an element of some resource type, consists of three stages:

- **Creation**. At some point the resource is created according to the rules set by the package that contains the definition of the resource's type. After creation, the resource is placed on the ledger in a cell owned by somebody.
- **Transaction(s)**. The resource might participate in transactions, if its owner wishes so. The owner might transfer his ownership of the resource freely, whereas programmatic interaction of the resource with other objects happens only according to the rules set by the defining package.
- **Destruction (optional)**. If the defining package permits it, the resource might at some point be destroyed. Examples of destructible resources are stablecoins that are backed by an off-chain reserve, where a coin gets destroyed when its off-chain collateral is removed from the treasury, and a wrapper for an asset bridged from another blockchain, which gets destroyed when the external asset ceases to be bridged.

The most important part of the lifecycle, in which most resources will spend almost all their life, is the phase when they can participate in transactions. An ordinary transaction (i.e. one which does not deploy a new package) consists of an arbitrary Axi program that transforms the state of the ledger, which we'll call the *transaction body*, and of some metadata, most importantly the *transaction originator*. This program, when executed, interacts with the ledger in three steps:

- **Withdraw**. First, resources are "withdrawn" from the ledger into the program. The ledger's ownership model ensures the safety of this step, for example by making sure that the program cannot withdraw resources that don't belong to the transaction originator.

- **Transform**. Second, the program transforms its resources by performing various computations, exchanging the resources for other resources and moving them around, etc. The safety of this step is guaranteed by the type system. The resources cannot get unlawfully destroyed or copied because the type system keeps an eye on them, nonsensical operations cannot be performed on them, because the program is well-typed, they cannot get lost in the jungle of pattern matching, because exhaustiveness is checked, and so on.

- **Deposit**. Third, the program "deposits" the transformed resources back onto the ledger. The safety of this step is partially ensured by the type system, which checks that all resources held by the program are deposited back. However, since resources can be freely transferred, it's up to the transaction author to make sure that they are transferred to the correct address and not to somebody else.

C.3 A longer example

Example 8. *Donut Shop*

```
module DonutShop where

// exporting the types would allow public access
// to constructors and record updates
// since the types are mentioned in the signatures
// the types will be public but opaque
export
  init
  buyDonut
  collectProfits
  getMyShopRef

// Donut - simple purchasable object, can be used at most once
record type Donut : Type? where
  serialNumber : Nat

// Donut shop - shared object
record type DonutShop : Type1 where
  thisShop : Address
  price : Nat
  balance : KhalaniCoin
  nextSerialNumber : Nat

record type DonutShopOwnershipToken : Type1 where
  myShopRef : Ref DonutShop

// Create a donut shop
init (ledger : Ledger)
: DonutShopOwnershipToken * Ledger
= let (shopRef, shopToLedger) = share ledger in
  let ownership : DonutShopOwnershipToken =
    record where
      myShopRef = shopRef in
```

```
  let shop : DonutShop = record where
    thisShop = addressof shopRef
    price = 1000
    balance = coinZero
    nextSerialNumber = 0 in
  (ownership, shopRef, shopToLedger shop)

// One can buy a donut for a set price
buyDonut (payment : KhalaniCoin) (shop : DonutShop)
: Option Donut * KhalaniCoin * DonutShop
= let (splitResult, change) = coinSplit shop.price payment in
  match splitResult with
  | none => (none, change, shop)
  | some paid =>
    let newBalance = coinMerge shop.balance paid in
    let newSerialNumber = shop.nextSerialNumber in
    let newShop : DonutShop =
      record shop where
        balance = newBalance
        nextSerialNumber = newSerialNumber + 1 in
    let donut : Donut = record where serialNumber = newSerialNumber in
    (some donut, change, newShop)

// Collects profits from shop (succeeds only for owner)
collectProfits (ownership : DonutShopOwnershipToken) (shop : DonutShop)
: KhalaniCoin * DonutShop
= if addressof ownership.myShop == shop.thisShop then
    let profits = shop.balance in
    let newShop = record shop with balance = coinZero in
    (profits, newShop, ledger)
  else
    (coinZero, shop, ledger)

// Reads shop reference from an ownership token
getMyShopRef (ownership : DonutShopOwnershipToken)
: DonutShop * DonutShopOwnershipToken
= (token.myShopRef, token)
```

```
from DonutShop import
  init

main (ledger : Ledger) : Ledger =
: Ledger -> Ledger
| ledger =>
  let (myAddress, ledger) = whoami ledger in
  let (ownershipToken, ledger) = init ledger in
  let (tokenAddress, initTokenCell) = freeze myAddress ledger in
  let ledger = initTokenCell ownershipToken in
  ledger
```

```
from DonutShop import
  DonutShop
  buyDonut

coinAddr
: Address
= 0x0664eebd187e0668e41e5d2236eaafe2d54dd9dbc76685e4ceac69c908c26446

shopAddr
: Address
= 0xf09785359712726892a2ec3b8b7cf460f0f6dd82e0bf5a7a8d2f2276035888cf

```

```
main
: Ledger -> Ledger
| ledger =>
  let (myAddress, ledger) = whoami ledger in
  let (optShopRef, ledger) = referenceof DonutShop shopAddr ledger in
  match optShopRef with
  | some shopRef =>
    let (optCoin, ledger) = delete KhalaniCoin coinAddr ledger in
    match optCoin with
    | some paymentCoin =>
      let (shop, returnShop) = update shopRef ledger in
      let (optDonut, changeCoin, shop) = buyDonut paymentCoin shop in
      let ledger = returnShop shop in
      let (newCoinAddr, initCoinCell) = new myAddress ledger in
      let ledger = initCoinCell changeCoin in
      match optDonut with
      | some donut =>
        let (newDonutAddr, initDonutCell) = new myAddress ledger in
        let ledger = initDonutCell donut in
        ledger
      | none => ledger
    | none => ledger
  | none => ledger
```

```
from DonutShop import
  DonutShopOwnershipToken
  collectProfits
  getMyShopRef

ownershipTokenAddr
: Address
= 0x22bf7f5182a7d22decc11f2a8fd94e2c6834cebd2ab2b239cab598987454bc1b

main
: Ledger -> Ledger
| ledger =>
  let (myAddress, ledger) = whoami ledger in
  let (optOwnershipToken, ledger) =
    delete DonutShopOwnershipToken ownershipTokenAddr ledger in
  match optOwnershipToken with
  | some ownershipToken =>
    let (shopRef, ownershipToken) = getMyShopRef ownershipToken in
    let (shop, returnShop) = update shopRef ledger in
    let (profitsCoin, shop) = collectProfits ownershipToken shop in
    let ledger = returnShop shop in
    let (newCoinAddr, initCoinCell) = new myAddress ledger in
    let ledger = initCoinCell profitsCoin in
    let (newTokenAddr, initTokenCell) = new myAddress ledger in
    let ledger = initTokenCell ownershipToken in
    ledger
  | none => ledger
```